can you write a simple MLP in PyTorch.

oss120

Thought for less than a second

Sure! Below is a **minimal, self-contained example** of a Multi-Layer Perceptron (MLP) built with PyTorch.
It includes:

1. **Model definition** (`SimpleMLP`)
2. **Synthetic dataset** (you can replace it with any real data, e.g. MNIST, tabular CSV, etc.)
3. **Training loop** (with loss, optimizer, and optional validation)
4. **Device-agnostic code** (CPU/GPU)

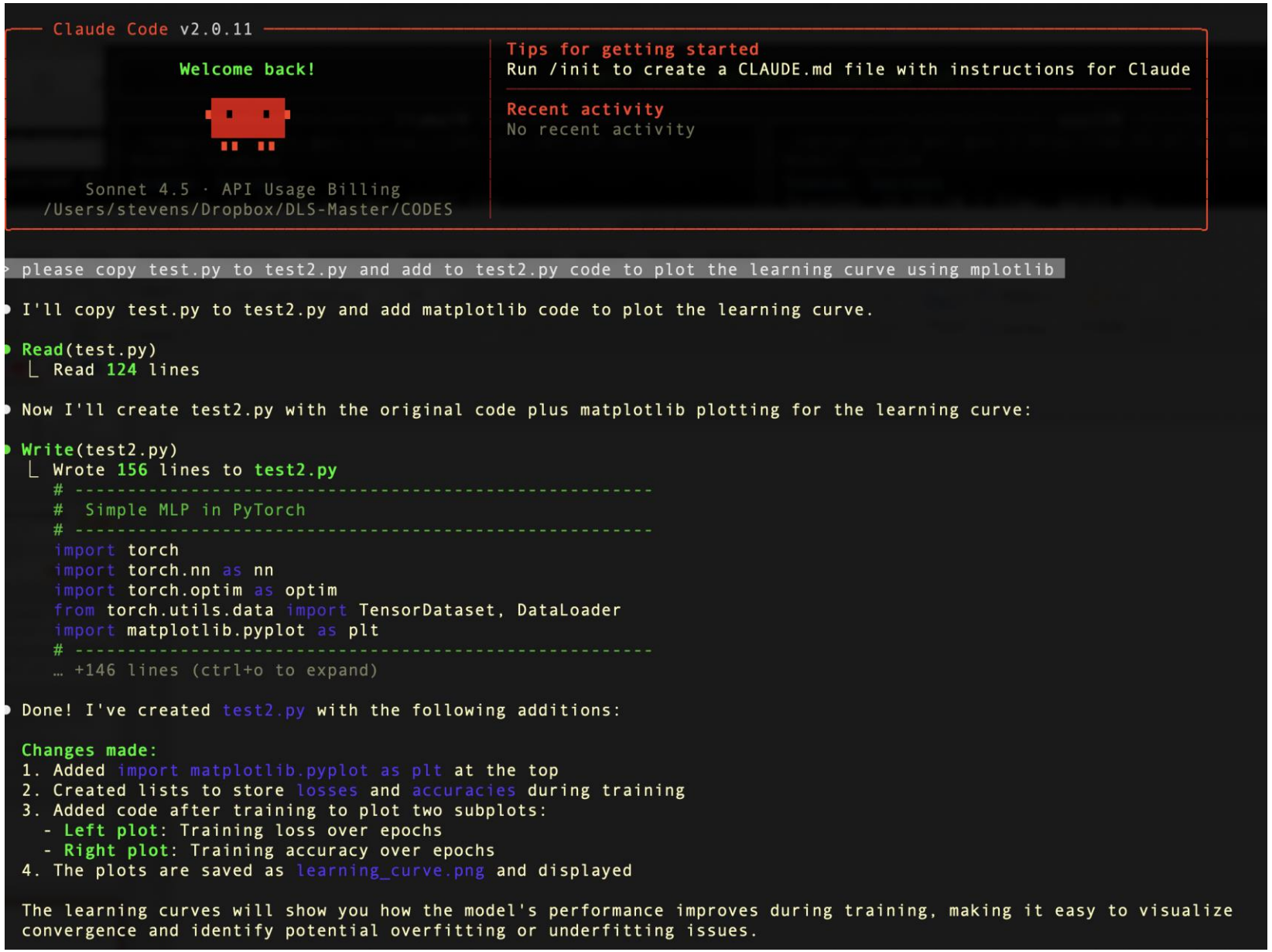
python Collapse Run Save Copy

```python
# ------------------------------------------------------------
#  Simple MLP in PyTorch
# ------------------------------------------------------------
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# ------------------------------------------------------------
#  1️⃣  Model definition
# ------------------------------------------------------------
class SimpleMLP(nn.Module):
    """A 3-layer fully-connected network."""
    def __init__(self, input_dim: int, hidden_dims: list[int], output_dim: int):
        """
        Args:
            input_dim  – dimensionality of the input features
            hidden_dims – list of hidden layer sizes, e.g. [64, 32]
            output_dim – number of output units (e.g. number of classes)
```

Send a Message

+ Code Interpreter

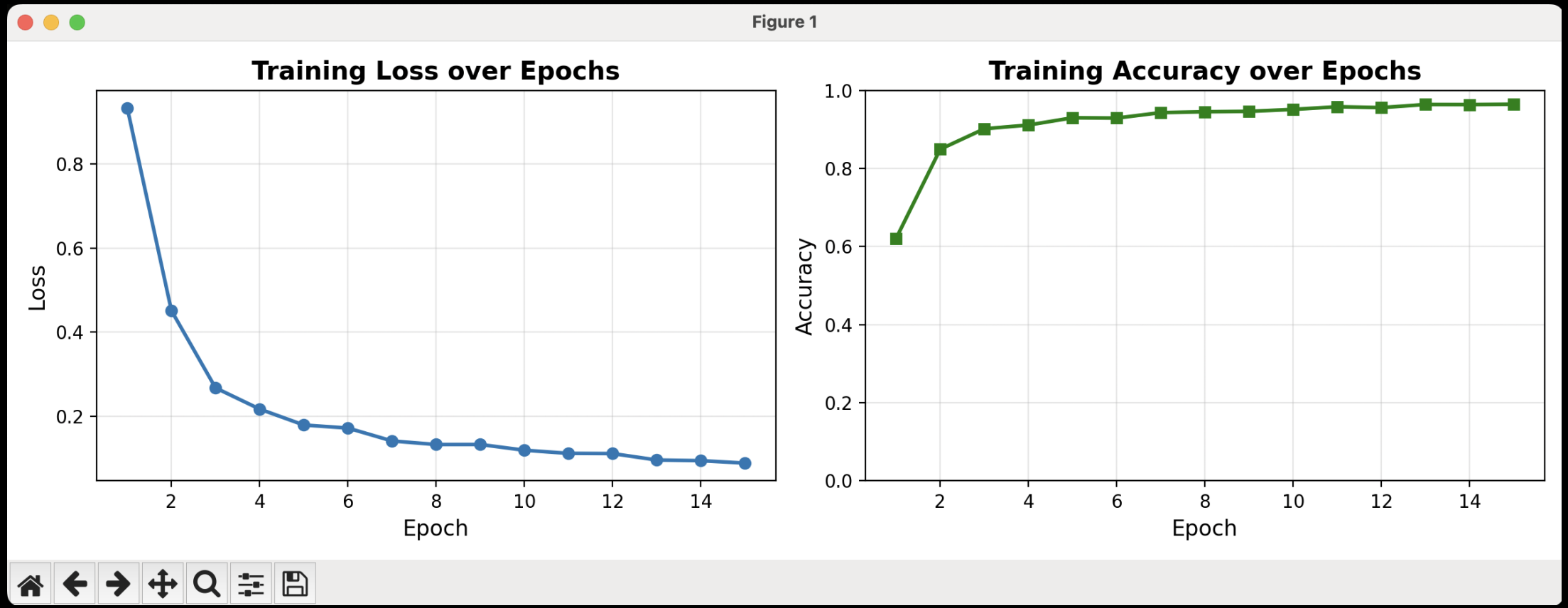

Claude Code v2.0.11
Welcome back!
Sonnet 4.5 · API Usage Billing
/Users/stevens/Dropbox/DLS-Master/CODES
Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude
Recent activity
No recent activity
> please copy test.py to test2.py and add to test2.py code to plot the learning curve using mplotlib
I'll copy test.py to test2.py and add matplotlib code to plot the learning curve.
Read(test.py)
└ Read 124 lines
Now I'll create test2.py with the original code plus matplotlib plotting for the learning curve:
Write(test2.py)
└ Wrote 156 lines to test2.py

Simple MLP in PyTorch
----
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import matplotlib.pyplot as plt

… +146 lines (ctrl+o to expand)
Done! I've created test2.py with the following additions:
Changes made:
1. Added import matplotlib.pyplot as plt at the top
2. Created lists to store losses and accuracies during training
3. Added code after training to plot two subplots:
- Left plot: Training loss over epochs
- Right plot: Training accuracy over epochs
4. The plots are saved as learning_curve.png and displayed
The learning curves will show you how the model's performance improves during training, making it easy to visualize convergence and identify potential overfitting or underfitting issues.

CODES — -zsh — zsh — Homebrew — 104×37

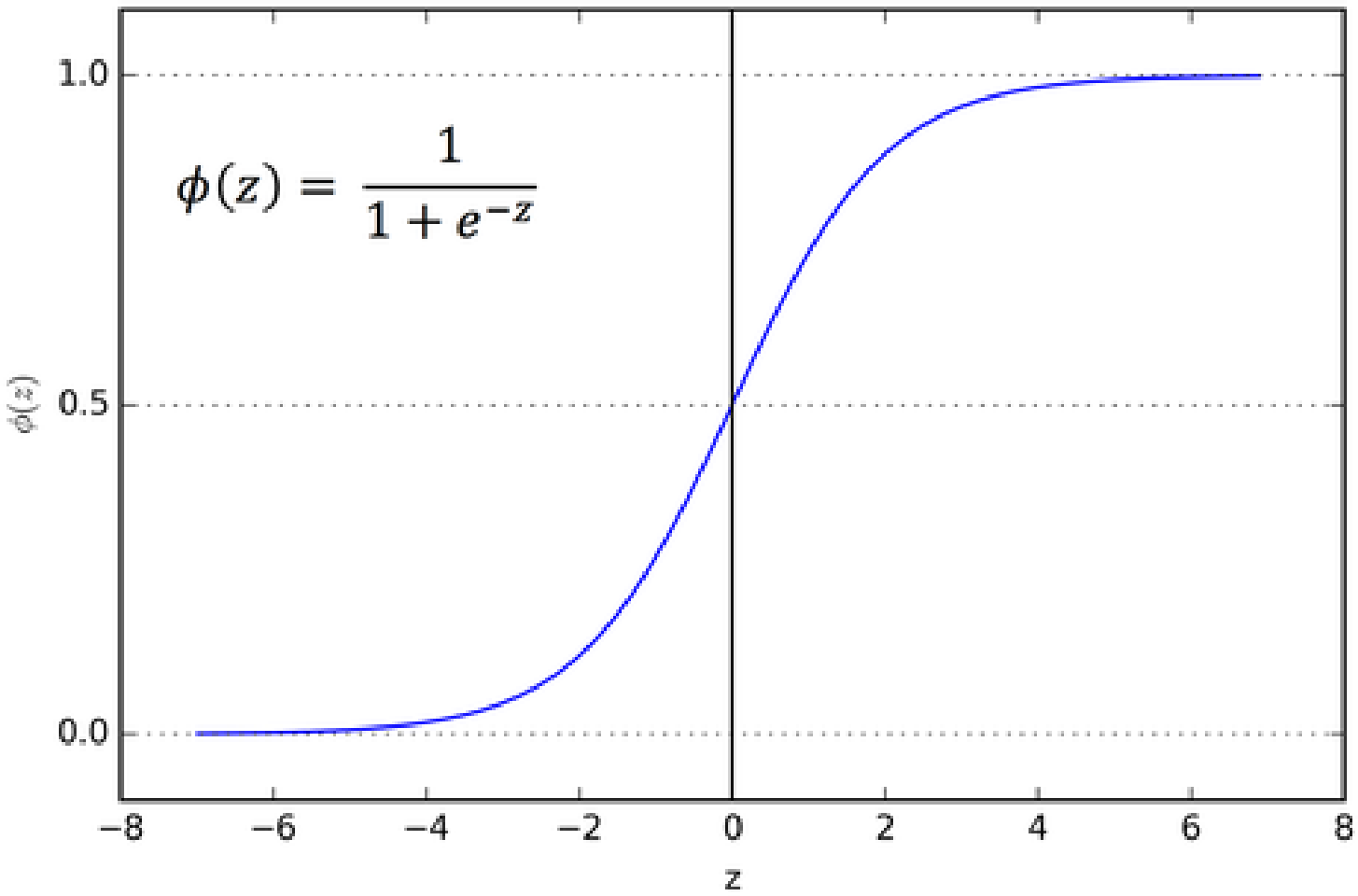
```
(base) stevens@M3acBook-Pro CODES % diff test.py test2.py
7a8
> import matplotlib.pyplot as plt
118a120,123
> # Store metrics for plotting
> losses = []
> accuracies = []
>
120a126,127
>     losses.append(loss)
>     accuracies.append(acc)
123a131,154
>
> # ------------------------------------------------------------
> #  6️⃣ Plot learning curves
> # ------------------------------------------------------------
> fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
>
> # Plot loss
> ax1.plot(range(1, EPOCHS + 1), losses, marker='o', linewidth=2, markersize=6)
> ax1.set_xlabel('Epoch', fontsize=12)
> ax1.set_ylabel('Loss', fontsize=12)
> ax1.set_title('Training Loss over Epochs', fontsize=14, fontweight='bold')
> ax1.grid(True, alpha=0.3)
>
> # Plot accuracy
> ax2.plot(range(1, EPOCHS + 1), accuracies, marker='s', color='green', linewidth=2, markersize=6)
> ax2.set_xlabel('Epoch', fontsize=12)
> ax2.set_ylabel('Accuracy', fontsize=12)
> ax2.set_title('Training Accuracy over Epochs', fontsize=14, fontweight='bold')
> ax2.grid(True, alpha=0.3)
> ax2.set_ylim([0, 1])
>
> plt.tight_layout()
> plt.savefig('learning_curve.png', dpi=150, bbox_inches='tight')
> print("\nLearning curves saved to 'learning_curve.png'")
(base) stevens@M3acBook-Pro CODES %
```


Figure 1
Training Loss over Epochs
Loss
Epoch
0.2
0.4
0.6
0.8
2
4
6
8
10
12
14
Training Accuracy over Epochs
Accuracy
Epoch
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
10
12
14

Activations

- **Originally “Squash” functions** – compressing values into a given interval
- **Differentiable** – we can compute the derivative
- **Monotonic** – entirely increasing or decreasing
- **Simple** – fast to compute

Sigmoid

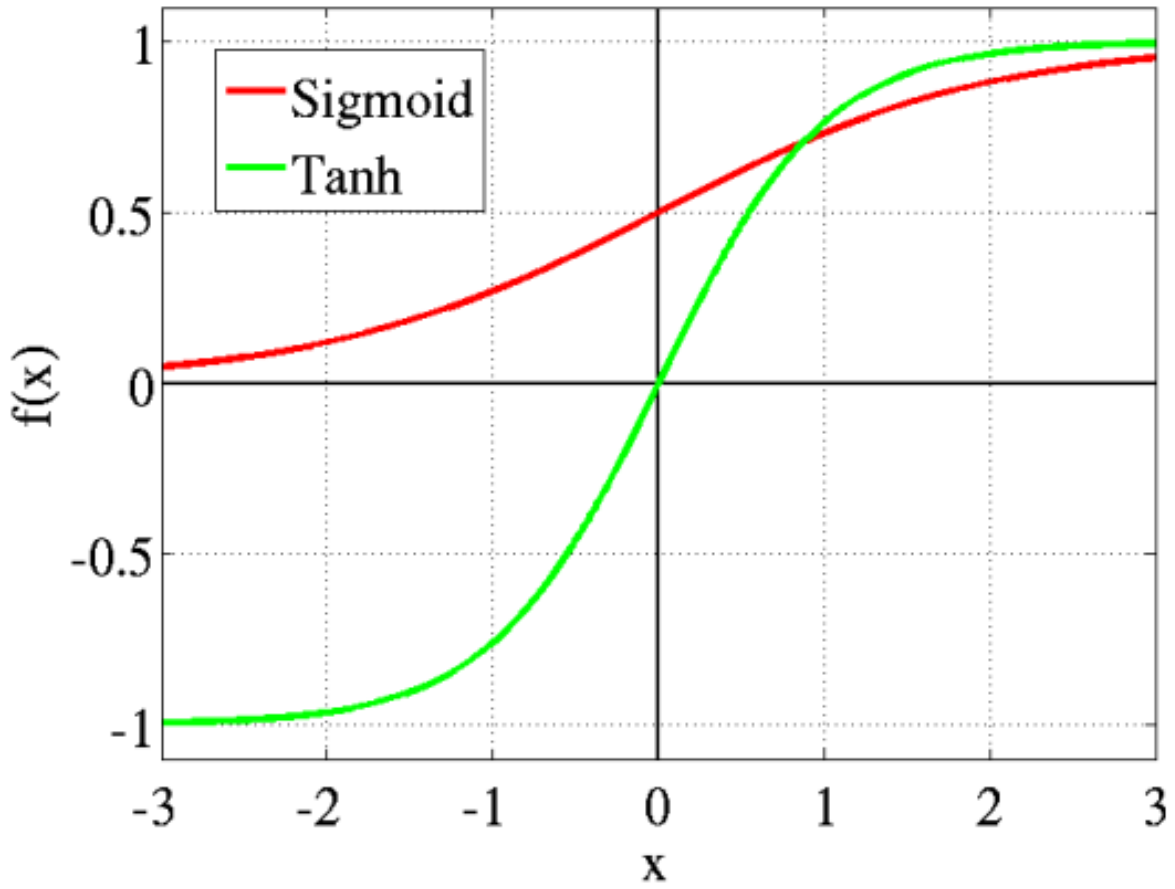


Sigmoid

- The main reason why we use sigmoid function is because it exists between **(0 to 1).** Therefore, it is especially used for models where we have to **predict the probability** as an output. Since probability of anything exists only between the range of **0 and 1,** sigmoid is the right choice
- The function is **differentiable**. That means, we can find the slope of the sigmoid curve at any point.
- The function is **monotonic** but the function's derivative is not.
- The logistic sigmoid function can cause a neural network to get stuck during training time
- The **softmax function** is a more generalized logistic activation function which is used for multiclass classification

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$

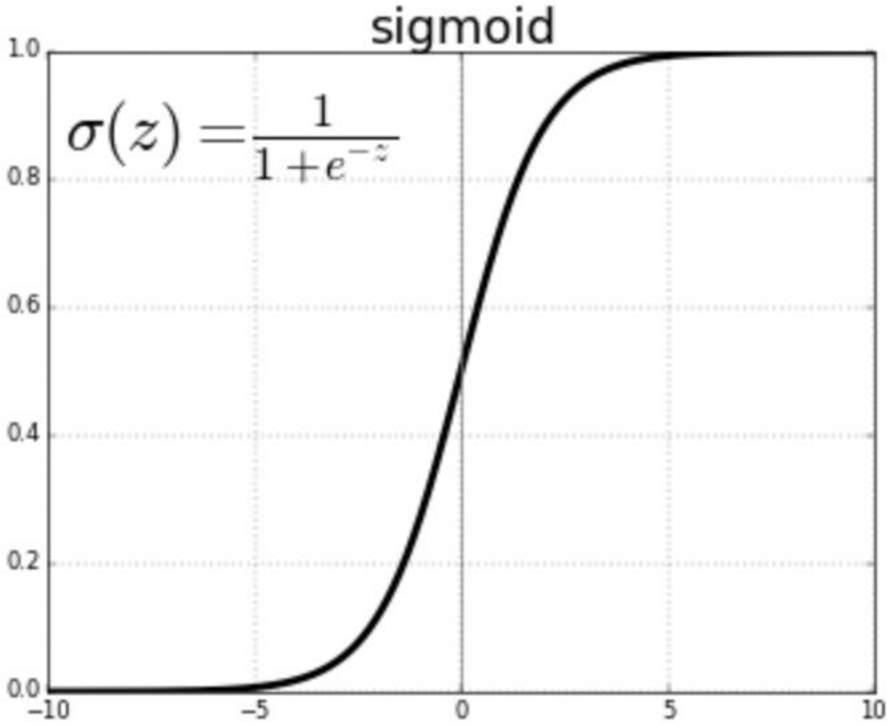
Tanh (hyperbolic tangent)

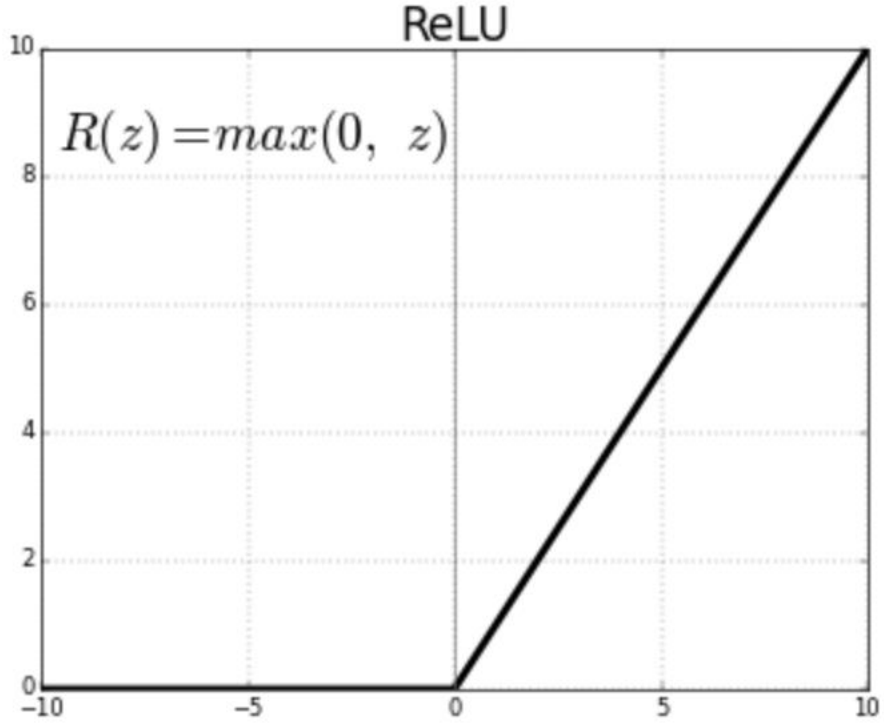


Tanh (hyperbolic tangent)

- Tanh is also like sigmoid but with -1 to 1 range
- Tanh is also sigmoidal (s - shaped)
- The advantage is that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph
- The function is **differentiable**
- The function is **monotonic** while its **derivative is not monotonic**
- The tanh function is mainly used in classification between two classes
- Both tanh and logistic sigmoid activation functions are used in feed-forward nets

ReLU (Rectified Linear Unit)

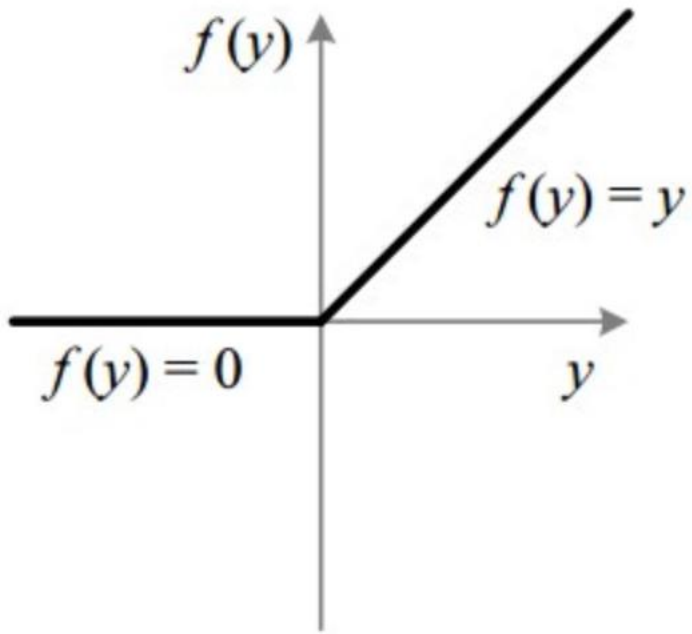


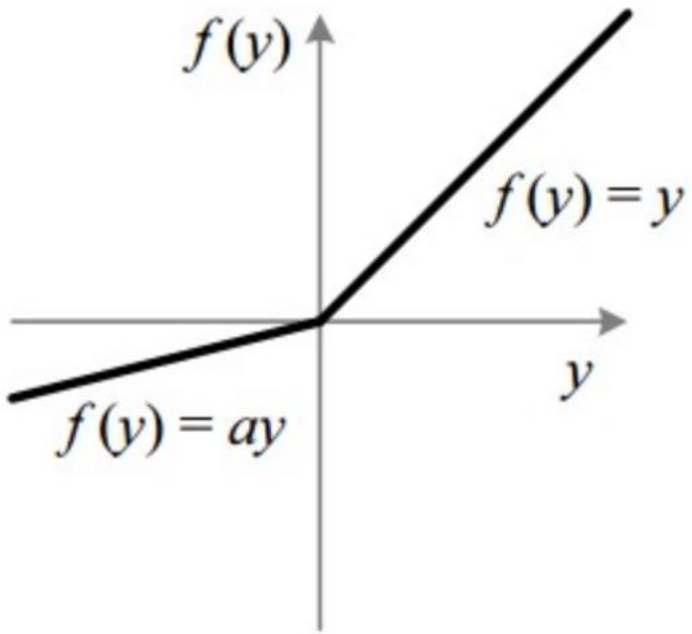


ReLU (Rectified Linear Unit)

- The ReLU is the most used activation function in the world right now. Since, it is used in almost all the convolutional neural networks in deep learning
- As you can see, the ReLU is half rectified (from bottom). f(z) is zero when z is less than zero and f(z) is equal to z when z is above or equal to zero.
- **Range:** [0 to infinity)
- The function and its derivative **both are monotonic**.
- But the issue is that all the negative values become zero immediately which can limit the ability of the model to fit or train from the data
- Negative input values given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

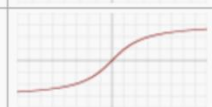
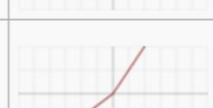
Leaky ReLU

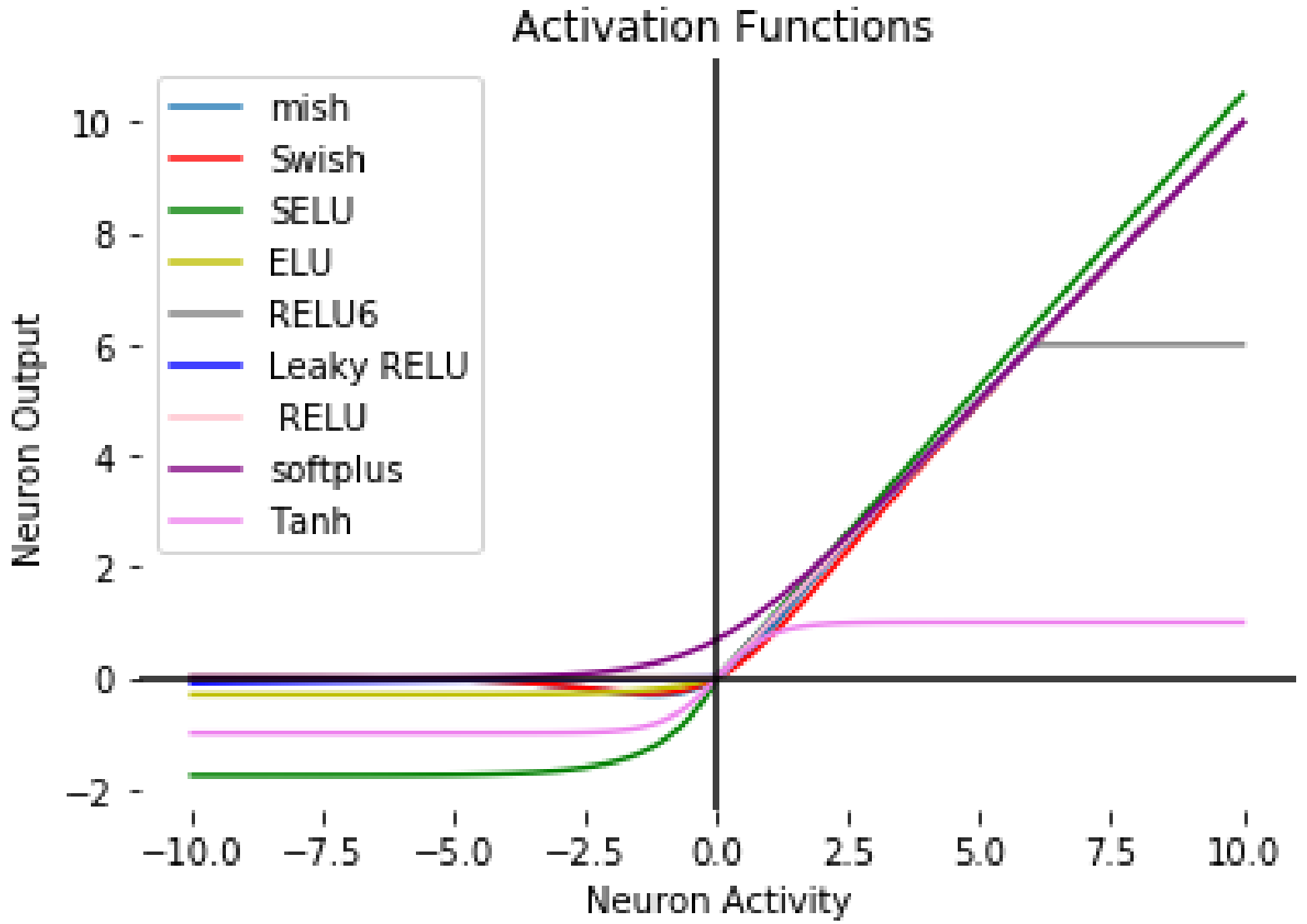




Leaky ReLU

- It is an attempt to solve the ReLU “zero” problem
- Can you see the Leak? 
- The leak helps to increase the range of the ReLU function. Usually, the value of **a** is 0.01 or so
- When **a is not 0.01** then it is called **Randomized ReLU**
- Therefore the **range** of the Leaky ReLU is (-infinity to infinity)
- Both Leaky and Randomized ReLU functions are monotonic, also, their derivatives also monotonic

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$


Activation Functions
mish
Swish
SELU
ELU
RELU6
Leaky RELU
RELU
softplus
Tanh
Neuron Output
10
8
6
4
2
0
−2
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
Neuron Activity

Baselines	ReLU	LReLU	PReLU	Softplus	ELU	SELU	GELU
Swish > Baseline	9	7	6	6	8	8	8
Swish = Baseline	0	1	3	2	0	1	1
Swish < Baseline	0	1	0	1	1	0	0

Table 3: The number of models on which Swish outperforms, is equivalent to, or underperforms each baseline activation function we compared against in our experiments.

Model	ResNet	WRN	DenseNet
LReLU	94.2	95.6	94.7
PReLU	94.1	95.1	94.5
Softplus	94.6	94.9	94.7
ELU	94.1	94.1	94.4
SELU	93.0	93.2	93.9
GELU	94.3	95.5	94.8
ReLU	93.8	95.3	94.8
Swish-1	94.7	95.5	94.8
Swish	94.5	95.5	94.8

Table 4: CIFAR-10 accuracy.

Model	ResNet	WRN	DenseNet
LReLU	74.2	78.0	83.3
PReLU	74.5	77.3	81.5
Softplus	76.0	78.4	83.7
ELU	75.0	76.0	80.6
SELU	73.2	74.3	80.8
GELU	74.7	78.0	83.8
ReLU	74.2	77.8	83.7
Swish-1	75.1	78.5	83.8
Swish	75.1	78.0	83.9

Table 5: CIFAR-100 accuracy.

$$f(x) = x \cdot \sigma(x)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = x \cdot \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$$

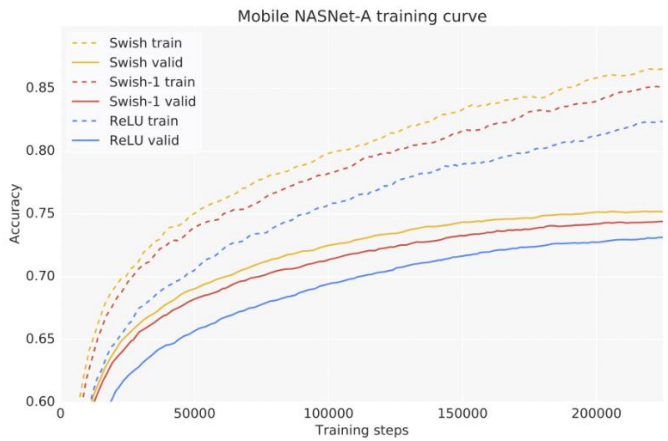


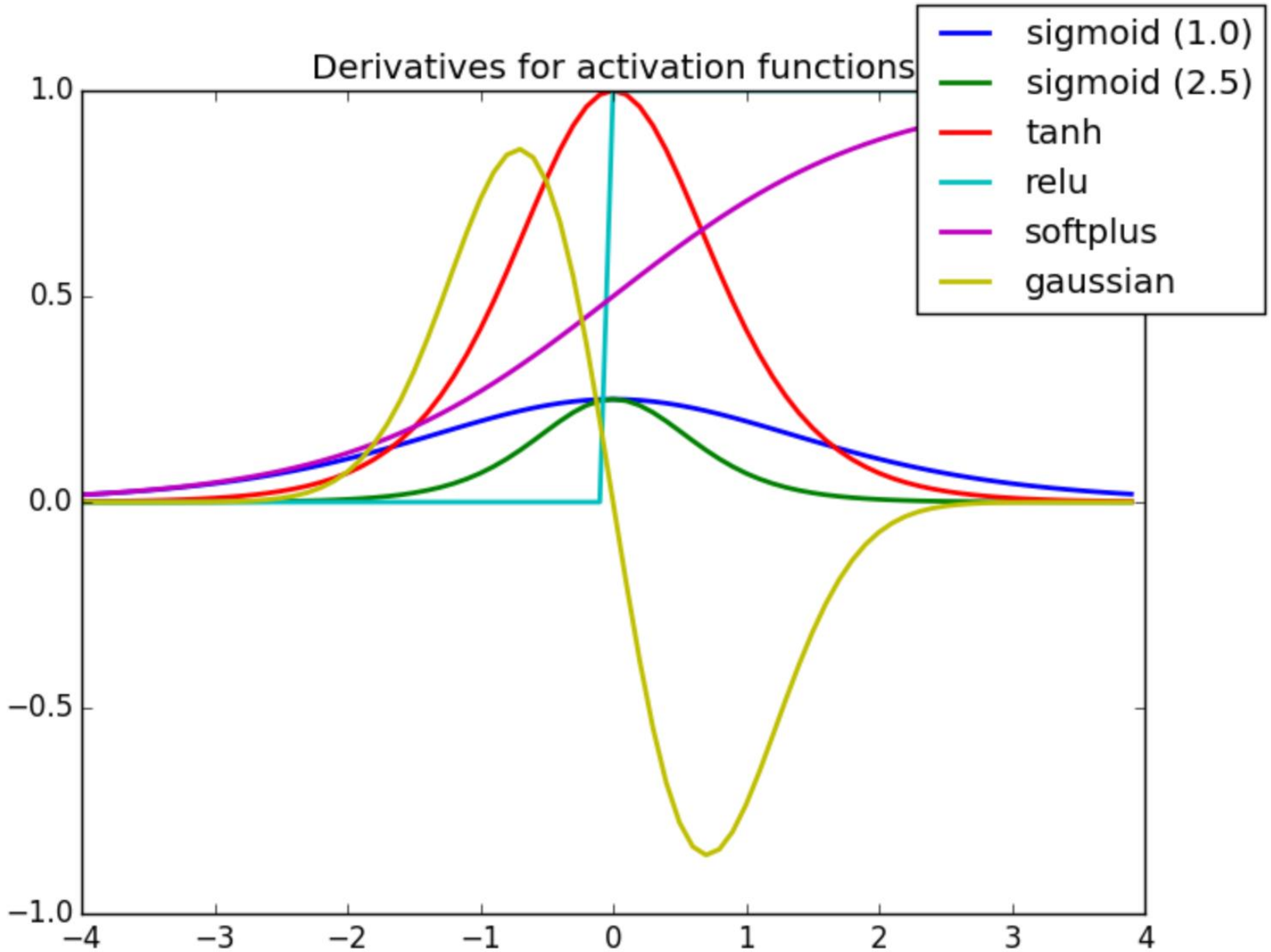
Figure 8: Training curves of Mobile NASNet-A on ImageNet. Best viewed in color

Model	Top-1 Acc. (%)			Top-5 Acc. (%)		
LReLU	73.8	73.9	74.2	91.6	91.9	91.9
PReLU	74.6	74.7	74.7	92.4	92.3	92.3
Softplus	74.0	74.2	74.2	91.6	91.8	91.9
ELU	74.1	74.2	74.2	91.8	91.8	91.8
SELU	73.6	73.7	73.7	91.6	91.7	91.7
GELU	74.6	-	-	92.0	-	-
ReLU	73.5	73.6	73.8	91.4	91.5	91.6
Swish-1	74.6	74.7	74.7	92.1	92.0	92.0
Swish	74.9	74.9	75.2	92.3	92.4	92.4

Table 6: Mobile NASNet-A on ImageNet, with 3 different runs ordered by top-1 accuracy. The additional 2 GELU experiments are still training at the time of submission.

Swish is the general form

SILU is with beta == 1


Derivatives for activation functions
sigmoid (1.0)
sigmoid (2.5)
tanh
relu
softplus
gaussian
1.0
0.5
0.0
−0.5
−1.0
−4
−3
−2
−1
0
1
2
3
4

Loss Functions

Definition

In a supervised deep learning context the **loss function** measures the **quality** of a particular set of parameters based on how well the output of the network **agrees** with the ground truth labels in the training data.

Nomenclature

loss function

=

cost function

=

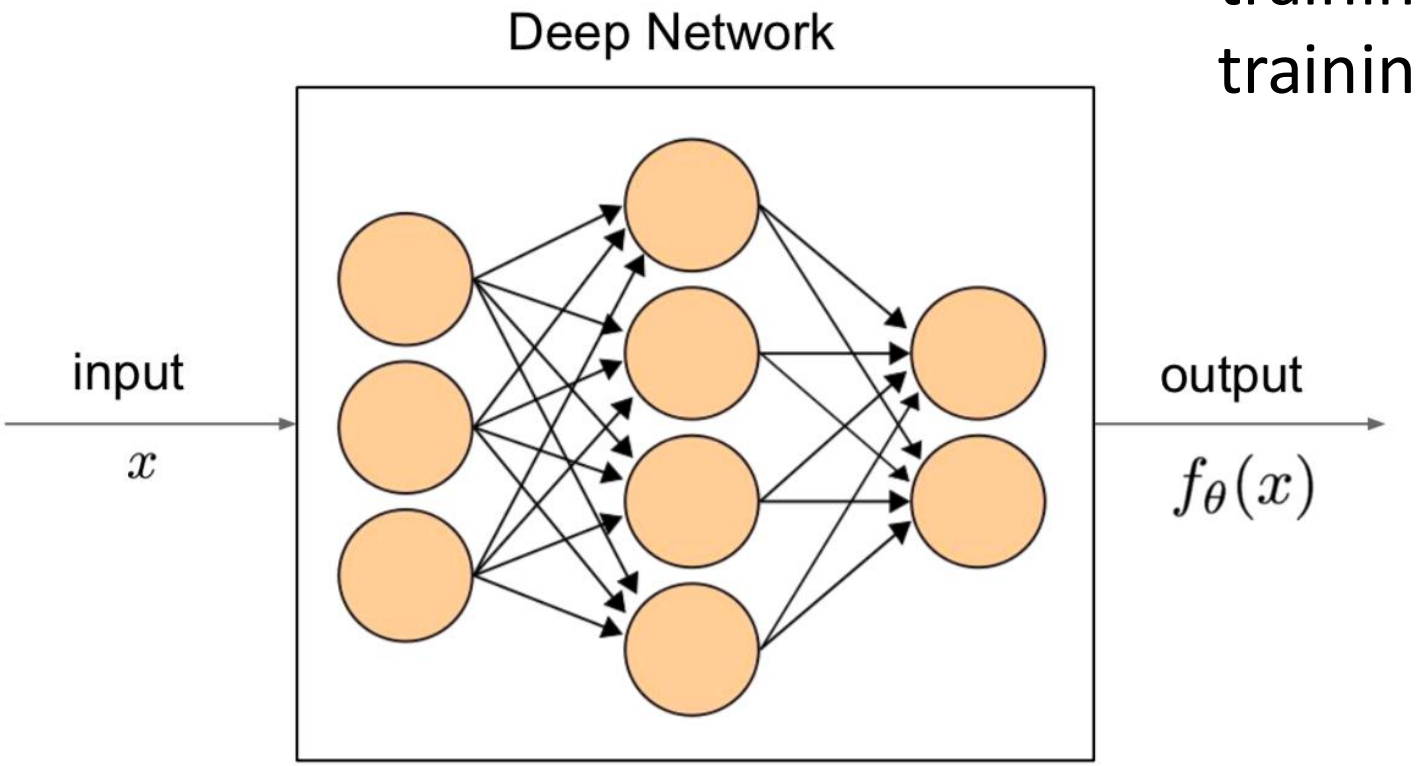
objective function

=

error function

Loss function

How close is the prediction of the model (trained from the training data) when compared to training data?



labels (ground truth)

input

$$\mathcal{L}(w) = distance(f_\theta(x), y)$$

error

parameters (weights, biases)

Training Loss vs Validation Loss (or Test Loss)

- The loss function is used to estimate the distance between the predictions the model makes and the labels in the training data
- This estimated distance (between what we are predicting and what the right answer is) or error is what will be used to improve the model

- The same function can be applied to compare the predictions of the model to labels in data not used to train the model (i.e. validation or testing data). This estimate is not used to improve the model directly and is referred to as "validation loss"
- Other metrics (perhaps more understandable to humans) might also be used to compare model predictions to validation data

Definition of Loss (MSE)

$$L(w,b) \equiv \frac{1}{2n} \sum_x \| \, y(x) - a \|^2$$

$y(x)$	The ground truth label for each sample *x*
a	The activation of the last layer for sample *x*

Loss Definition

$$L(w) \equiv \frac{1}{2n} \sum_{x} \| y - \hat{y} \|^2$$

y is the label for sample *x* and *y* (hat) is the prediction for sample *x*

we also collapse the bias into the weights for simplicity of presentation

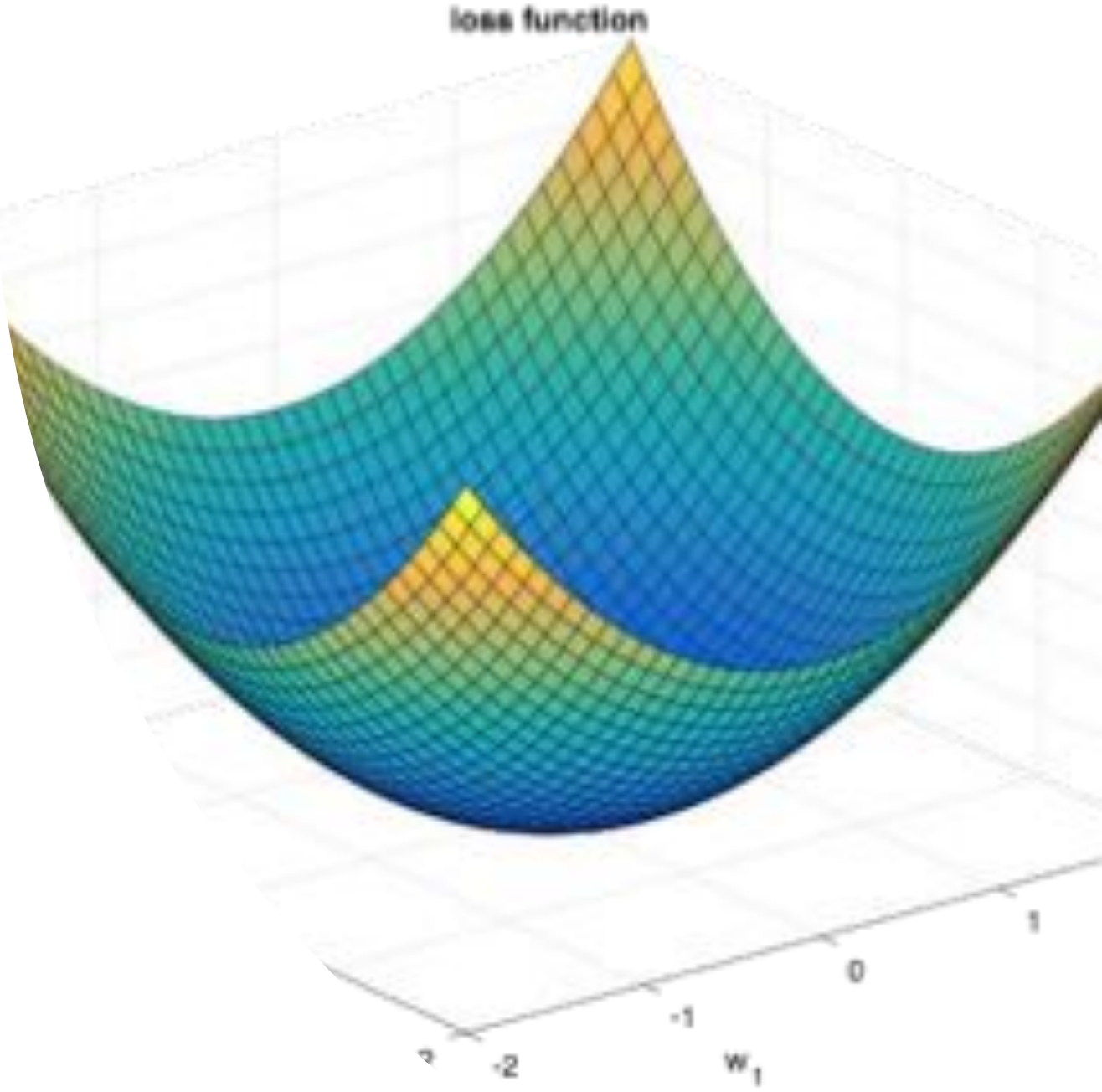
Overall Loss is Mean Loss over Samples

$$L = \frac{1}{n} \sum_{x} L_x$$

We simplify our notation to just refer to Loss and Loss for a specific sample

Properties we need in Loss Function

- Minimum (0 value) when the output of the network is equal to the ground truth data
- Increase value when output differs from ground truth
- Ideally $\Longrightarrow$ convex function
- In reality $\Longrightarrow$ (order millions of parameters and not convex)
- Varies smooth with changes in the output
  - Gradients for gradient descent
  - Easy to compute small changes in parameters to get an improvement in loss



Assumptions

- For **backpropagation** to work:
  - Loss function can be written as an average over loss functions for individual training examples:

empirical risk

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_i$$

  - Loss functions can be written as a function of the output activations from the neural network.

Loss Functions depend on task type

- **Regression:** predict continuous numeric values
  - Absolute Error, Squared Error, etc.

- **Classification:** predict categorical values
  - Hinge, Cross-entropy, etc.

Gradient of the Loss

The gradient of the loss is equal to the mean of the gradient over each sample loss

$$\nabla L = \frac{1}{n} \sum_{x} \nabla L_x$$

This fact will be useful when we are trying to estimate the overall loss gradient from subset of samples

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors[2]. Learning becomes more interesting but more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_j, to unit j is a linear function of the outputs, y_i, of the units that are connected to j and of the weights, w_{ji}, on these connections

$$x_j = \sum_i y_i w_{ji} \tag{1}$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

A unit has a real-valued output, y_j, which is a non-linear function of its total input

$$y_j = \frac{1}{1+e^{-x_j}} \tag{2}$$

At its essence backpropagation is a clever application of the chain rule

The chain rule is a fundamental property of derivatives taught in any undergraduate curriculum. It states that if you have 3 functions *f*, *g* and *h* with *f* being a function of *g* and *g* being a function of *h* then the derivative of *f* with respect to *h* is equal to the product of the derivative of *f* with respect to *g* and the derivative of *g* with respect to *h.* This can be neatly written as:

$$f(g(h(x)))$$

$$\frac{df}{dh} = \frac{df}{dg}\frac{dg}{dh}$$